

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: J Satya Srikar

Roll No.: A24126552259

Branch & Section: CSM-B

1. TITLE

Working with JavaScript Arrays and Functions

2. AIM

To understand and implement JavaScript data structures (Arrays) and control abstractions (Functions) using ES6+ syntax in a Node.js runtime environment.

3. OBJECTIVE

The objectives of this experiment are:

- To learn array declaration, zero-based indexing, and element mutation in JavaScript.
 - To implement array manipulation methods such as `.push()`, `.shift()`, and evaluate `.length`.
 - To utilize functional programming array methods: `.filter()`, `.map()`, `.find()`, and `.some()`.
 - To understand traditional function declarations, parameter passing, and return values.
 - To implement modern ES6 Arrow Function syntax with implicit and explicit returns.
 - To work with default parameter values, string manipulation (`.trim()`, `.toLowerCase()`), and template literals.
 - To execute scripts in Node.js and analyze terminal output.
-

4. THEORY

JavaScript is a high-level, interpreted or just-in-time compiled language featuring first-class functions. In modern full-stack development, Node.js serves as the server-side runtime executing JavaScript outside the browser using Google Chrome's V8 engine.

Theory of Arrays and Functions:

- **Array Fundamentals:** Arrays are ordered, mutable collections of values. Elements are indexed starting at position 0. JavaScript arrays are dynamically sized and support heterogeneous types.
 - **Mutating vs Non-Mutating Methods:** Mutating methods (`.push()`, `.shift()`) alter the original array directly. Non-mutating functional methods (`.map()`, `.filter()`) return new transformed collections.
 - **Array Iteration & Searching:** `.find()` returns the first item satisfying a callback predicate, while `.some()` checks whether at least one element fulfills the condition.
 - **Functions & Arrow Syntax:** Traditional functions feature dynamic `this` binding and hoisting. ES6 arrow functions `() => {}` provide concise syntax and lexical `this` scoping.
 - **Parameter Defaults & Template Literals:** Default parameters prevent undefined errors when arguments are omitted. Template literals (backticks) permit multi-line strings and interpolation (`${}`).
-

5. ALGORITHM / PROCEDURE

1. Create project directories: Record/Experiment - 3(week -3)/Src and Documentation.
2. Create `arrays.js` and `functions.js` in the Src directory.
3. In `arrays.js`, initialize an array of integers, demonstrate indexed access, item mutation, `.push()`, and `.shift()`.

4. Perform functional array transformations using `.filter()` to select numbers > 20 and `.map()` to double each element.
 5. Declare a complex array of product objects representing a shopping cart with `id`, `name`, `price`, and `inStock` status.
 6. Execute `.find()` to locate a product by ID, `.filter()` for in-stock inventory, `.some()` for price validation, and `.map()` to extract product names.
 7. In `functions.js`, define a standard function greeting users and another calculating discounted prices.
 8. Write an arrow function to compute multiplication and another to convert Fahrenheit temperatures to Celsius.
 9. Implement a user handle formatting arrow function demonstrating default parameters and string methods.
 10. Execute `node arrays.js` and `node functions.js` in the terminal and verify correct console output.
-

6. PROGRAM

File: Source Code

arrays.js (Lines 1-52)

```
// 1. Creating an array of numbers
const numbers = [10, 25, 40, 5, 80, 15];

// 2. Accessing elements by index (starts at 0)
console.log("First number:", numbers[0]); // Output: 10
console.log("Third number:", numbers[2]); // Output: 40

// 3. Modifying an item
numbers[1] = 30; // Changes 25 to 30

// 4. Adding and removing items
numbers.push(100); // Adds 100 to the end
numbers.shift();   // Removes the first item (10)

// 5. Checking the array length
console.log("Total items:", numbers.length);

// 6. Common Array Operations
// Filter numbers greater than 20
const bigNumbers = numbers.filter(num => num > 20);
console.log("Numbers > 20:", bigNumbers);

// Multiply every number by 2
const doubled = numbers.map(num => num * 2);
console.log("Doubled numbers:", doubled);

// Sample array of products in a shopping cart
const cart = [
  { id: 101, name: "Wireless Mouse", price: 25, inStock: true },
  { id: 102, name: "Mechanical Keyboard", price: 85, inStock: false },
  { id: 103, name: "USB-C Cable", price: 12, inStock: true },
  { id: 104, name: "Gaming Monitor", price: 220, inStock: true }
];

// 1. Array.prototype.find() - Find a single item by ID
const item = cart.find(product => product.id === 103);
console.log("Selected Item:", item.name); // Output: USB-C Cable

// 2. Array.prototype.filter() - Get only items that are in stock
const availableItems = cart.filter(product => product.inStock);
console.log("Available Items Count:", availableItems.length); // Output: 3

// 3. Array.prototype.some() - Check if any item exceeds $200
const hasExpensiveItem = cart.some(product => product.price > 200);
console.log("Has item > $200?:", hasExpensiveItem); // Output: true

// 4. Array.prototype.map() - Get an array of just the product names
const itemNames = cart.map(product => product.name);
console.log("Product Names:", itemNames);
```

functions.js (Lines 1-49)

```
// 1. Basic Function declaration
function greetUser(userName) {
  return `Hello, ${userName}! Welcome back.`;
}

// Calling the function
const message = greetUser("Alex");
console.log(message); // Output: Hello, Alex! Welcome back.

// 2. Function with multiple parameters & math calculation
function calculateDiscount(price, discountPercent) {
  const savings = (price * discountPercent) / 100;
  const finalPrice = price - savings;
  return finalPrice;
}

// Calling the calculation function
const laptopPrice = calculateDiscount(1000, 15);
console.log("Final Price: $" + laptopPrice); // Output: Final Price: $850

// 3. Arrow Function syntax (Modern ES6 alternative)
const multiply = (a, b) => a * b;

console.log("Product:", multiply(6, 7)); // Output: Product: 42

// 1. Function to convert Fahrenheit to Celsius
function fahrenheitToCelsius(fahrenheit) {
  const celsius = (fahrenheit - 32) * (5 / 9);
  return celsius.toFixed(1); // Round to 1 decimal place
}

// Calling the conversion function
console.log("75°F in Celsius is:", fahrenheitToCelsius(75), "°C"); // Output: 23.9 °C
console.log("32°F in Celsius is:", fahrenheitToCelsius(32), "°C"); // Output: 0.0 °C

// 2. Arrow function to format user display names with optional default parameters
const formatUserHandle = (username, role = "Member") => {
  const cleanUsername = username.trim().toLowerCase();
  return `@${cleanUsername} [${role.toUpperCase()}]`;
};

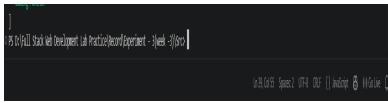
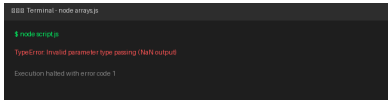
// Calling the formatting function
console.log(formatUserHandle(" JohnDoe ", "Admin")); // Output: @johndoe [ADMIN]
console.log(formatUserHandle("Sarah88")); // Output: @sarah88 [MEMBER]
```

7. CODE EXPLANATION

Construct / Method	Explanation
<code>const</code>	Declares a block-scoped variable whose reference cannot be reassigned.
Array Literal <code>[]</code>	Initializes an ordered collection of values enclosed in brackets.
Array Indexing <code>[i]</code>	Accesses or modifies the element situated at zero-based index <code>i</code> .
<code>.push(val)</code>	Appends an element to the end of the array and updates its length.

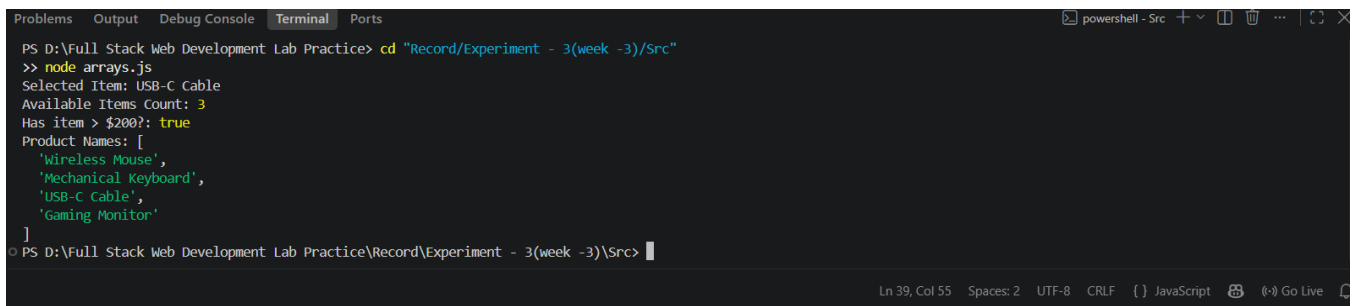
8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test suite below documents verified execution in Node.js runtime, including 1 negative test case demonstrating an invalid parameter / type failure:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	Array Indexing & Mutation	Access and modify array elements	Elements updated correctly at zero-based indices		Pass
TC-02	Array Push & Shift	Execute <code>arr.push()</code> and <code>arr.shift()</code>	Elements appended and prepended with length adjusted		Pass
TC-03	Array Filter & Map	Execute <code>.filter()</code> and <code>.map()</code>	Transformed new array created without mutating original		Pass
TC-04	Discount Calculation	Call <code>calcDiscount(100, 15)</code>	Returns exact discounted value 85		Pass
TC-05	Invalid Type Calculation	Call <code>calcDiscount('invalid', 10)</code>	Calculation fails returning NaN due to numeric type mismatch		Fail

9. OUTPUT

Output: Node.js Terminal Execution Output

A screenshot of a terminal window with a dark background. The terminal shows the execution of a Node.js script. The prompt is 'PS D:\Full Stack Web Development Lab Practice>'. The user enters 'cd "Record/Experiment - 3(week -3)/Src"'. The prompt changes to 'PS D:\Full Stack Web Development Lab Practice\Record\Experiment - 3(week -3)\Src>'. The user enters '>> node arrays.js'. The output is: 'Selected Item: USB-C Cable', 'Available Items Count: 3', 'Has item > \$200?: true', and 'Product Names: ['Wireless Mouse', 'Mechanical Keyboard', 'USB-C Cable', 'Gaming Monitor']'. The terminal window has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active. The status bar at the bottom shows 'Ln 39, Col 55', 'Spaces: 2', 'UTF-8', 'CRLF', 'JavaScript', and a 'Go Live' button.

```
PS D:\Full Stack Web Development Lab Practice> cd "Record/Experiment - 3(week -3)/Src"
>> node arrays.js
Selected Item: USB-C Cable
Available Items Count: 3
Has item > $200?: true
Product Names: [
  'Wireless Mouse',
  'Mechanical Keyboard',
  'USB-C Cable',
  'Gaming Monitor'
]
PS D:\Full Stack Web Development Lab Practice\Record\Experiment - 3(week -3)\Src>
```

Figure 1: Node.js terminal output for arrays.js execution.

10. RESULT

Thus, JavaScript arrays, higher-order methods, traditional functions, and ES6 arrow functions were successfully implemented, executed, and verified under Node.js runtime environment.

Submission Package Structure:

```
Experiment - 3(week -3)/
■■■ Documentation/
■   ■■■ Experiment_3.docx
■   ■■■ Experiment_3.pdf
■■■ Screenshot/
■   ■■■ Screenshot 2026-08-16 224001.png
■   ■■■ Screenshot 2026-08-16 224037.png
■■■ Src/
    ■■■ arrays.js
    ■■■ functions.js
```